ECJ's Warts

ECJ has been around for almost eight years, for much of the length of Java's public existence. Though ECJ was overengineered and overwrought from the start, this overengineering has granted it a great deal of flexibility and the toolkit has, I think, proven unusually adaptable.

But ECJ has a lot of warts, many of which could be cleaned up with some work, though I'm worried about backward-compatability. Keep in mind that ECJ design was begun in 1997. For those of you old enough to remember :-), Java 1.1 was released in February 1997 -- and it was buggy -- and Java 1.2 wasn't released until December 1998. Lots of installations were still using Java 1.0. JITs were only just coming out, and HotSpot didn't exist. At that time, Java had a number of missing features and efficiency problems which influenced ECJ's design.

Here are some warts I've identified. Send me mail if you'd like some others discussed here.

1. **Setup and Prototype.** ECJ needed a way to make many copies of objects whose classes were specified dynamically. There was only one real way to do this: Class.forName(name).newInstance(). As newInstance() only called the default constructor, there was no reasonable way (at the time) to use a constructor with multiple arguments in order to pass it parameter information. Thus setup(...) was born.

   setup(...) in most objects is fairly complex, consisting of a lot of checks to make sure everything is kosher. These checks aren't necessary for quick hacks -- I wrote them into ECJ because it's library code and should be relatively bulletproof. But unfortunately they lead people to thinking that setup(...) methods need to be complicated, when they can be quite trivial in fact.

   ECJ also needed to make large numbers of copies of objects; but the clone() method was protected and could not be unprotected in an interface. Thus clone() was used to copy objects off of Prototypes.

   To make matters even more complex, clone() was originally conceived without a definition as to whether it was a deep clone or not. This is problematic because GPTrees in some cases need to be light-cloned and sometimes deep-cloned. So Individual has a deep-clone method as well as a clone() method. Instead we should have made ALL clones deep, and instead created a "light clone" method for GPIndividual.

   In versions of ECJ prior to 15, clone() was originally called protoClone(), and there was also a protoCloneSimple() which called protoClone() wrapped in a try { ...} catching CloneNotSupportedException. There's some historical discussion in the CHANGES file, but this code is gone now. It's all clone now.

2. **Group, Singleton, and Clique.** Group, a variant of Prototype was meant for objects for which there'd be no prototypical object hanging around in storage (but instead it'd be actively used). It turned out that this was only used for Population and Subpopulation, which could probably have been folded into Prototype without much effort (just make new arrays on clone()).

   Cliques proved to be useful but not as an interface. Quite a number of ECJ objects are Cliques, and they all follow the same pattern: a central repository which is globally accessible, storing a small N number of objects. This repository should have been moved into Clique itself, making Clique a class. But it wasn't. Additionally, the repository was a global (a static variable). This has prevented ECJ from being entirely modularized until recently, when we moved all these variables into Initializers for lack of a better place to put them.

Singletons never had a reason to be an Interface -- it's totally empty and there's nothing special about them. Ultimately we might delete them and just have them be Setups.

3. **Parameter and ParameterDatabase.** Parameters have always been wrappers for Strings. Why weren't they Strings to begin with? Because I was over-engineering, and worried that Strings couldn't be subclassed. As it turns out, Parameters have never needed to be anything other than Strings. Perhaps ParameterDatabase could be reengineered to allow Strings as well as Parameters.

   ParameterDatabase sits on top of PropertyList rather than XML. For good reason: XML didn't exist at the time. But I don't view this as a wart really: it's a feature. XML was never meant to be a data transfer format, and its misuse as one produces huge amounts of typing and syntactic errors. PropertyLists are much simpler and I'm glad I went with them. The disadvantage of PropertyLists is that they're flat. As a result, you see huge, long dotted parameters like "pop.subpop.0.species.pipe.source.0 = ec.gp.koza.CrossoverPipeline". A tree format like XML would solve this, sort of.

4. **Output and Code.** This logging facility is directly influenced by lil-gp. At the time, Java had no good logging facility at all (and one was only introduced in 1.4.2). The goal of Output was primarily to provide messages which could be repeated after restoring from checkpoint, and it does this quite nicely. But other features of Output have worn less well. Multiple verbosity levels in particular was a feature of Output which I have *never* used, and I think it just complicates things. Making matters worse, some functions have the the log (an integer) first and the verbosity (an integer) next, whereas other functions do it the other way around. Definitely a historical stupidity on my part. Also, I do not believe I've ever used the facility for writing to all logs at one time.

   I wound up using Output's logging to write Individuals out to stdout and to various statistics files. I think this was an error. Later on the need would arise to write Individuals out to Writers and read them from Readers anyway. As a result Individuals all have methods for writing Individuals out to streams AND to log them in Output.

   For a long time, ECJ has had two different ways for writing individuals out: one which is human-readable only, and one which can be read by humans and by machines. This allows GPIndividuals to be dumped in a "pretty" format and a "read-in" format. To pull off the trick of a human AND computer readable format, I constructed an encoding mechanism for numbers which included both their human-readable format and raw bits. It's called Code. A Coded double (2.932341) looks like this:

   ```
   d4613785463717479022|2.932341|
   ```

   Perhaps this might be better described as "human-decodable" rather than "human-readable". It worked well but parsing back in is slow (fine for files, bad for sockets). And the reading mechanism (a DecodeReturn tokenizer) is overly complex. Use of Code, Output, and other gunk makes ECJ's printFoo and readFoo methods quite difficult to understand.

   The original code for island models used printIndividual and readIndividual to send individuals across streams. This is grotesquely inefficient, necessitating the creation of lots of Strings and lots of parsing. A much better approach would be to use DataInputStream and DataOutputStream, but I was concerned that this would result in even MORE ways to print and read Individuals. Well, we're going to bite the bullet and do it, for island models and the client/server evaluator at least.

5. **Quicksort and SortComparator.** Why are these here? Simple: Collections didn't even exist yet, to say nothing of the Arrays class. There *was* no sorting procedure available. We probably should get rid of this one day.

6. **floats.** ECJ uses floats internally for nearly everything rather than doubles. This is mostly a copy from lil-gp; but at the time (and still now frankly) floats were rather faster than doubles.

7. **VectorIndividuals.** This package has a lot of subclasses, one for each atomic type. Java 1.5 will allow templating and you'd think that this would allow us to clean things up. Well, it would, but at the cost of speed. A VectorIndividual or whatnot actually would be stored internally as a VectorIndividual of Integer instances, which is highly inefficient. Templating in Java 1.5 is little more than sugar coating, which is a real shame. So... not a wart yet.

8. **final.** Lots of things in ECJ were (or still are) final. This is because final used to make a VERY big difference in speed as JITs figured out where they could inline. Nowadays (1.4 and on) final has no effect at all. But you still see a lot of things declared final for no really good reason. Sometimes these don't harm anything (such as final arguments). Sometimes they prevent subclassing -- I've removed most examples of the latter.

9. **GP.** There are oodles of warts here:
   - GPNode.eval has tons of arguments and no clean way to extend them. Instead of:

```
public abstract void eval(final EvolutionState state,
                          final int thread,
                          final GPData input,
                          final ADFStack stack,
                          final GPIndividual individual,
                          final Problem problem);
```

     ... it should have been something like

```
public abstract void eval(final GPData input,
                          final GPContext context);
```

   - GPTreeConstraints and GPNodeConstraints really don't do much; both could have been folded into GPIndividual and GPFunctionSet.
   - The koza package probably should have been named something else. :-) I'm sure John's a bit weirded out by his name being used like that.
   - koza/CrossoverPipeline and koza/MutationPipeline should have been renamed to something more consistent with other names. They're too general terms.
   - GPAtomicType could have been folded into GPSetType, or both into GPType (defined as set types). There's no real need for GPAtomicType.

10. **Rules.** The rule package has been woefully underutilized. I had expected to use it a lot, but so far haven't much.

11. **Fitness.** Our definition of Fitness was highly vague on purpose. Is a high number better? Is a low number better? Are there more than one number? Not defined. But this vagueness tends to cause problems for algorithms which need to have Fitness defined in one specific way or

another. Notably, KozaFitness and SimpleFitness both define fitness radically differently internally; but must be both usable in FitProportionateSelection.